Comprehensions in Python

All four comprehension types, their patterns and decision rules on one page.

The four types

Type	Syntax	Example
list	`[e for x in it]`	`[x * 2 for x in xs]`
set	`{e for x in it}`	`{x % 3 for x in xs}`
dict	`{k: v for x in it}`	`{x: x ** 2 for x in xs}`
generator	`(e for x in it)`	`sum(x for x in xs)`

General grammar = nested loops, flattened

```
[ EXPR  for A in X  if C1  for B in Y  if C2 ]
```

```
out = [(a, b) for a in range(3)
       for b in range(3) if a < b]
same = []
for a in range(3):
    for b in range(3):
        if a < b:
            same.append((a, b))
print(out == same, out)
```

```
OUTPUT
True [(0, 1), (0, 2), (1, 2)]
```

Decision guide

You need...	Use
ordered, reusable results	list comprehension
unique items / fast `in` tests	set comprehension
a key → value mapping	dict comprehension
one pass over big or endless data	generator expression
side effects (print / write / send)	plain `for` loop

Pattern gallery

```
xs = [3, -1, 4, -1, 5]
print([x for x in xs if x > 0])          # filter
print([abs(x) for x in xs])              # map
print(sorted({x for x in xs}))           # unique
print({x: xs.count(x) for x in xs})      # count
# dedupe, keep order
print(list(dict.fromkeys(xs)))
```

```
OUTPUT
[3, 4, 5]
[3, 1, 4, 1, 5]
[-1, 3, 4, 5]
{3: 1, -1: 2, 4: 1, 5: 1}
[3, -1, 4, 5]
```

Nested comprehensions

```
grid = [[r * c for c in range(1, 4)]
        for r in range(1, 4)]
print(grid)
print([x for row in grid for x in row if x % 2 == 0])
d = {"a": 3, "b": 1, "c": 2}
print(dict(sorted(d.items(), key=lambda kv: kv[1])))
```

```
OUTPUT
[[1, 2, 3], [2, 4, 6], [3, 6, 9]]
[2, 2, 4, 6, 6]
{'b': 1, 'c': 2, 'a': 3}
```

Move logic into a helper

```
def clean(s):
    return s.strip().lower()

print([clean(s) for s in ["  Ann ", "BOB  "]])
```

```
OUTPUT
['ann', 'bob']
```

Comprehension vs map / filter vs loop

Style	Notes
`for` + `append`	most flexible; needed for side effects
comprehension	idiomatic, compact, usually faster
`map` / `filter`	fine when you already have a named function
generator expression	lazy; use for one-pass streams

Gotchas

- `{}` is an empty **dict** – write `set()` for an empty set.
- No tuple comprehension: use `tuple(x for x in it)`.
- Comprehensions have their **own scope** (loop variable doesn't leak).
- Nested deeper than two levels? Switch to loops – readability wins.

TIP Readability test: if you can't read it aloud in one breath, use a loop or a helper function.